

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Virtuální nástěnka pro podporu výuky na 1. stupni
základních škol



2025

Vedoucí práce:
RNDr. Martin Trnečka, Ph.D.

Michal Šmajzr

Studijní program: Informační technologie,
prezenční forma

Bibliografické údaje

Autor: Michal Šmajzr
Název práce: Virtuální nástěnka pro podporu výuky na 1. stupni základních škol
Typ práce: bakalářská práce
Pracoviště: Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby: 2025
Studijní program: Informační technologie, prezenční forma
Vedoucí práce: RNDr. Martin Trnečka, Ph.D.
Počet stran: 37
Přílohy: elektronická data v úložišti katedry informatiky
Jazyk práce: český

Bibliographic info

Author: Michal Šmajzr
Title: Virtual bulletin board to support teaching in the 1st grade of primary schools
Thesis type: bachelor thesis
Department: Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense: 2025
Study program: Information Technologies, full-time form
Supervisor: RNDr. Martin Trnečka, Ph.D.
Page count: 37
Supplements: electronic data in the storage of department of computer science
Thesis language: Czech

Anotace

Tato práce se zabývá vytvořením webové aplikace, která se bude snadno používat učitelům na 1. stupni základních škol. Pro vytvoření webové aplikace jsou využity technologie jako jsou framework Next.js, Tailwind CSS a databáze MySQL, které umožňují vytvořit moderní online nástroj, který je do budoucna rozšiřitelný. Nástroj bude mít využití na základních školách, táborech, kroužcích nebo skupinových lekcích a umožňuje učiteli vytvářet nástěnky, sekce, příspěvky, ankety a posílat zprávy mezi učitelem a uživatelem.

Synopsis

This bachelor thesis deals with the creation of a web application that will be easy to use for teachers at the 1st level of primary schools. To create the web application, I used the latest technologies such as the Next.js framework, Tailwind CSS and the MySQL database, which allowed me to create a modern online tool that is expandable in the future. The tool will be used in primary schools, camps, clubs or group lessons and allows the teacher to create bulletin boards, sections, posts, polls and send messages between the teacher and the user.

Klíčová slova: virtuální nástěnka; uživatelské rozhraní; REST API; Next.js; Tailwind CSS; MySQL;

Keywords: virtual bulletin board; user interface; REST API; Next.js; Tailwind CSS; MySQL;

Rád bych poděkoval svému vedoucímu práce RNDr. Martinu Trnečkovi, Ph.D. za jeho odborné rady.

Odevzdáním tohoto textu jeho autor/ka místopřísežně prohlašuje, že celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

Obsah

1	Výběr technologií	9
1.1	HTML	9
1.2	CSS	9
1.3	Tailwind CSS	9
1.4	JavaScript	9
1.5	TypeScript	9
1.6	React	10
1.7	Node.js	10
1.8	Next.js	10
1.9	MySQL	11
2	Architektura	12
2.1	Modulární architektura	12
2.2	REST API	12
3	Implementace	12
3.1	Návrh UI/UE	12
3.2	Návrh databáze	14
3.2.1	Transakce	14
3.2.2	Tabulky uživatelů	14
3.2.3	Tabulky nástěnek	16
3.2.4	Tabulky dotazů	16
3.2.5	Tabulky anket	17
3.2.6	Tabulky a záznamy pro odznaky (badges)	17
3.3	Autentizace	17
3.4	Implementace komponent	18
3.5	Implementace REST API	18
3.5.1	Struktura	18
3.5.2	Databáze a typování	19
3.5.3	Odpovědi	19
3.5.4	Odchycení chyb	19
3.5.5	Zabezpečení	20
3.5.6	Poskytnutí dat	20
3.6	Získávání dat	20
3.7	Posílání, mazání a změna dat	21
3.8	Nástěnky, sekce, příspěvky	21
3.8.1	Nástěnky a sekce	21
3.8.2	Příspěvky	22
3.8.3	Textové příspěvky	22
3.8.4	Fotografie	22
3.9	Komprese	23
3.10	Vyhledávání	23
3.11	Dotazy	23

3.12	Ankety	24
3.13	Správa uživatelů	25
3.13.1	Vytvoření uživatele	26
3.14	Nastavení uživatele	27
3.15	Mobilní verze	27
3.16	Bezpečnost	28
3.17	Rozšíření	29
4	Porovnání s nástrojem Padlet	30
	Závěr	31
	Conclusions	32
A	První příloha	33
B	Druhá příloha	33
C	Obsah elektronických dat	33
	Literatura	35

Seznam obrázků

1	Stránka s nástěnkami	13
2	EER diagram databáze	15
3	Stránka s dotazy	24
4	Dialogové okno s výsledky na stránce ankety	25
5	Stránka se správou uživatelů	25
6	Mobilní verze	28

Úvod

V dnešní době existuje mnoho online nástrojů, které zastávají různé funkce, ale pro výuku převážně ve školství nejsou zcela vyhovující. Diskutovali jsme s učiteli na základních školách a zjistili jsme, že jsou přehlčeni vším, co musí používat. Dnešních online nástrojů je mnoho a učitelé i rodiče se v nich ztrácí. Od školy dostanou nástroj pro nahrávání dokumentů, jsou nuceni komunikovat přes e-mail a používat externí nástroje na zpětnou vazbu. Proto se v této práci zabýváme, jak implementovat webovou aplikaci, která vše sjednotí a bude se snadno používat nejen učitelům, ale také rodičům.

Hodně z těchto aplikací se také soustředí výhradně na funkčnost a často ne navrhne dobře uživatelské rozhraní, aplikace sice pak funguje dobře, ale uživatelé často takové aplikace nechtějí používat. Proto se v práci také věnujeme návrhu dobrého a moderního designu a také uživatelského rozhraní, které se bude snadno používat.

Často se také stává, že se napíše aplikace, která se časem musí přepisovat do jiné technologie, protože již není možné aplikaci dále rozšiřovat. Vybrali jsme proto pro naši aplikaci nejnovější technologie, které nám zajistí dlouhou podporu a také nám dají možnosti, jak aplikaci do budoucna rozšiřovat a škálovat.

V první kapitole pojednáváme o základních technologiích, které jsme použili, a proč jsme je zvolili. Z nich se nejvíce budeme věnovat frameworku Next.js. Druhou kapitolu jsme věnovali architektuře webové aplikace, kde si povíme, co je to modulární architektura a REST API. Třetí kapitola je nejrozsáhlejší a věnujeme se v ní samotné implementaci, nejdříve se zabýváme návrhem uživatelského rozhraní, poté návrhem databáze a jednotlivých tabulek. Projdeme si také autentizaci, implementaci komponent a REST API. Budeme se zabývat posíláním dat mezi klientem a serverem a také si ukážeme, jak implementovat nástěnky, sekce a příspěvky včetně vyhledávání. V části také budeme mluvit o dotazech, anketách a správě uživatelů. Na konci se budeme věnovat responzivnímu designu a bezpečnosti. V čtvrté kapitole srovnáme náš online nástroj s existující alternativou jako je Padlet a v závěru prezentujeme výsledky, kterých jsem dosáhli.

1 Výběr technologií

Výběr technologií je naprosto zásadní, často volíme podle požadavků, funkce aplikace a také možné rozšiřitelnosti do budoucna.

1.1 HTML

HTML (HyperText Markup Language) je hypertextový značkovací jazyk, který udává základní strukturu a sémantiku webu. Popisuje strukturu dokumentu a přes hypertextové odkazy propojuje jednotlivé dokumenty. Základním prvkem je element, který se skládá ze značek a může obsahovat atributy.

1.2 CSS

CSS (Cascading Style Sheets), česky kaskádové styly, slouží k vizualizaci HTML elementů. Jednotlivé styly bývá časem náročné udržovat, proto se využívají různé konvence a přístupy. Jeden z nich je atomický design [1], každou vlastnost můžeme reprezentovat právě jednou třídou a díky tomu se udržuje nízká specifičnost a znovupoužitelnost tříd.

1.3 Tailwind CSS

Jeden z nejznámějších a profesionálních frameworků je utility first framework Tailwind CSS [2]. Ten implementuje atomický design [1], kde právě každá jedna utilita reprezentuje jednu třídu. Všechny utility se zapisují přímo do HTML, a není proto nutné mít mnoho souborů pro styly. Jeho aktuální nejnovější verze 4 byla použita pro psaní uživatelského rozhraní, protože umožňuje vysokou flexibilitu a udržitelnost.

1.4 JavaScript

JavaScript je skriptovací jazyk, který slouží pro skriptování na straně klienta a umožňuje psát interaktivní webové stránky. Dnes se využívá jeho standard ECMAScript.

1.5 TypeScript

JavaScript je dynamicky typovaný jazyk, což umožňuje měnit typ proměnných za běhu programu, ale zároveň to může způsobovat chyby. Proto jsem se rozhodl využít TypeScript [3], který do Javacriptu přidává statické typování a tím zvyšuje čitelnost kódu a snadněji se předchází chybám.

1.6 React

React [4] je knihovna, která zjednodušuje psaní uživatelského rozhraní. Umožňuje psát znovupoužitelné komponenty, které mohou přijímat vlastnosti a udržují si svůj vnitřní čas. Funkční komponenty mohou používat React Hooky, což jsou funkce, které umožňují manipulovat s daty.

1.7 Node.js

Node.js [5] je framework, pomocí kterého lze psát plnohodnotný webový backend v JavaScriptu. Obsahuje moduly, kde každý modul zajišťuje určitou funkcionalitu. Další výhody jsou asynchronní zpracování a vysoká škálovatelnost.

1.8 Next.js

Next.js [6] je open source framework od Vercelu postavený na Reactu a umožňuje klientský, statický i server side rendering. Má v sobě zabudovaný Node.js server, proto je v něm možné psát plnohodnotný backend. Já jsem použil jeho aktuální nejnovější verzi 15, která přináší nový směrovač App Router.

Mezi hlavními složkami a soubory bychom vystihli následující:

- `public` – tato složka slouží pro uložení statických obrázků, které jsou následně dostupné přes URL, používáme ji pro ikony, které ukládáme do složky `icons`. Dále se zde nachází výchozí `favicon.ico`, která se zobrazuje v záložce webového prohlížeče a nahrazujeme ji vlastní ikonkou.
- `src` – při instalaci jsme zvolili možnost s vytvořením složky `src`, která může v budoucnu sloužit pro další rozšiřování aplikace. Pokud se tato možnost nezvolí, vytvoří se zde přímo složka `app`.
- `app` – slouží jako výchozí složka pro směrování v aplikaci. Každá vytvořená složka v `app` je dostupná přes URL a definuje danou trasu. Lze tedy vytvářet hierarchickou strukturu složek podle požadavků projektu, což je velmi přehledné a snadno se spravuje a rozšiřuje. V každé složce se nachází stránka `page.tsx` pro komponenty dané stránky a může se nacházet `layout.tsx`, kde lze definovat layout pro danou stránku a podstránky dané trasy. Také zde lze definovat dynamické trasy, které se vepisují do hranatých závorek, například `[userId]`. Jméno uvnitř závorek se pak používá k získání daného segmentu. Složku `api` v `app` lze strukturovat stejným stylem a umožňuje vytvářet API endpointy s názvem `route.ts`.
- `middleware.ts` – slouží pro provedení kódu ještě před dokončením požadavku. Používáme pro zabezpečení přístupu ke stránkám.
- `.env (.env.local)` – slouží pro uložení konfiguračních dat. Máme zde definovaných několik proměnných jako `NEXTAUTH_SECRET` (tajný klíč

k zabezpečení JWT tokenů), `NEXTAUTH_URL` (adresa k autentizaci) a `BASE_URL` (URL adresa stránky).

Dále jsme vytvořili několik složek pro další uspořádání projektu:

- `private` – složku jsme vytvořili pro soubory a obrázky, které ukládáme za běhu programu. Zpřístupňujeme je přes zabezpečené API endpointy.
- `components` – do složky `components` ukládáme jednotlivé komponenty. Pro jejich pojmenování jsme využili pascal case konvenci.
- `lib` – pro ukládání pomocných a konfiguračních souborů jsme vytvořili složku `lib`. Ukládáme zde konfiguraci k databázi v souboru `db.ts`, pomocnou funkci knihovny `NextAuth.js` `auth` [7] v `auth.ts` a konfiguraci pro odesílání mailů v souboru `mailer.ts`.
- `types` – tuto složku jsme přidali do struktury, abychom měli jednotné místo, kam ukládat rozhraní a typy v Typescriptu.
- `styles` – pro styly jsme vytvořili složku `styles`, kde ukládáme soubor `globals.css`. Mám zde uloženu definici veškerých barev, které jsme vygenerovali dle Material design 3 [8] pomocí Material Theme Builder [9]. Také jsme zde dle stejné dokumentace definovali velikosti písma, zaoblení a stíny. Jako výchozí písmo jsme použili Roboto z Google fonts. Toto písmo máme definované jako proměnnou v `globals.css` a v `layout` `RootLayout` jsme písmo importovali a přidali konfiguraci jako jsou tloušťky (400, 500, 700), styly (normal, italic) a znakovou sadu (latin-ext). Písmo nemusí být dostupné ihned, proto máme nastavený `display: "swap"`, který načte záložní písmo dané výchozím prohlížečem a ve chvíli, co bude písmo stažené, se přepne.

Next.js [6] disponuje dvěma základními principy vykreslování komponent. Prvním z nich je server side rendering, kdy se komponenty vykreslí na serveru a uživateli se pošle již hotová stránka, v případě druhé možnosti se komponenty vykreslují na klientovi. V defaultním nastavení se všechny stránky vykreslují na serveru s výjimkou stránek, kde je uvedeno `"use client"`, které se vykreslí na klientu. Výhoda server side renderingu je, že vše zpracuje server, jistá nevýhoda je, že v server komponentách nelze použít React hooky, jako jsou například `useState`, `useEffect` a z toho důvodu spíše upřednostňujeme klientské komponenty, abychom mohli vytvářet interaktivní aplikaci.

1.9 MySQL

Z povahy dat bylo nejvýhodnější zvolit relační databázi, vybrali jsme proto MySQL. K databázi je vždy dobré zvolit vhodné uživatelské rozhraní, které zjednodušuje její správu a vytváření dotazů, vybrali jsme proto `phpmyadmin`, alternativně bychom mohli použít například `adminer`.

2 Architektura

Architektura webové aplikace je základní struktura a popisuje, jak mezi sebou jednotlivé části aplikace spolupracují a interagují. [10]

2.1 Modulární architektura

Pro napsání webové aplikace v Next.js se nám nabízely dvě možnosti, první z nich byla použít monolitickou architekturu (všechno v jednom) a použít pro psaní webového backendu Server Action (což jsou asynchronní operace, které běží v Node.js a lze je použít přímo v komponentách pro zpracování dat) nebo použít modulární architekturu s REST API. Rozhodli jsme se pro druhou možnost především kvůli rozšiřitelnosti samotné aplikace, jednotlivé API endpointy nám umožňují se neomezovat pouze na naši aplikaci, ale poskytovat je dále. Budeme moci tedy v budoucnu aplikaci dále rozšiřovat a implementovat nativní mobilní či desktopovou aplikaci. Aplikace se tedy skládá z webového front-endu, REST API a databáze. Každý API endpoint je modul, který zajišťuje danou funkcionalitu, komunikuje s databází a vrací klientovi data.

2.2 REST API

REST API, v překladu aplikační programové rozhraní založené na architektuře REST, bylo pro nás ideální řešení. Komunikace probíhá pomocí čtyř základních HTTP metod GET, POST, PUT, DELETE a každá z nich má svůj specifický účel: GET slouží pro získávání dat, POST pro jejich posílání, PUT pro změnu dat a DELETE slouží pro mazání dat. Pokrývají tedy všechny základní operace, které se provádí s daty nad databází.

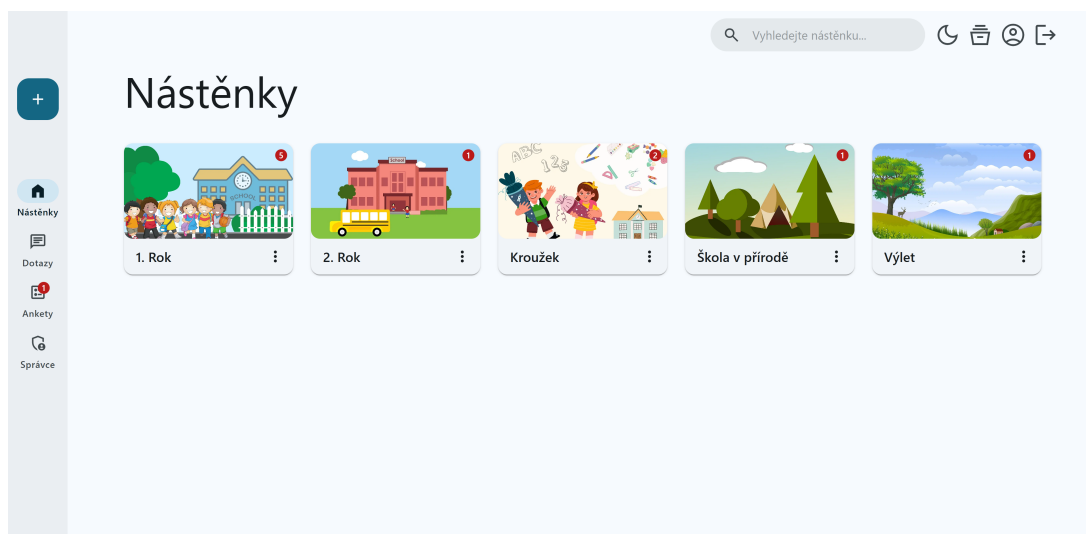
Celé to tedy funguje následovně. Ve webové aplikaci se používá klasická klient server architektura. Klient, v našem případě webový prohlížeč, se dotazuje přes HTTP požadavek serveru, což jsou v našem případě API endpointy, které běží v Next.js (interně používá Node.js server) a ty obsluhují daný požadavek. API endpoint daný požadavek zpracuje, provede interakci s databází, od které získá data a tyto data pošle zpět v HTTP odpovědi klientovi, který je ve webovém prohlížeči vykreslí. [11]

3 Implementace

V této kapitole se zabýváme implementací webové aplikace.

3.1 Návrh UI/UE

Před samotným napsáním webové aplikace je vždy důležité navrhnout uživatelské rozhraní. User interface se zabývá tím, jak webová stránka vypadá a user experience se zabývá rozložením prvku, aby se aplikace dobře používala. Pro



Obrázek 1: Stránka s nástěnkami

uživatelsky přívětivou stránku je nutné se zabývat oběma styly. Pro návrh se využívají designové systémy, které obsahují jednotlivé vzory, z kterých se bude uživatelské rozhraní skládat. Pro návrh komplexního designu je potřeba navrhnout převážně jednotlivé komponenty, barvy, typografii, ikony a layout (rozvržení). Rozhodli jsme se použít Material Design 3 [8] od Googlu, který obsahuje většinu těchto vzorů, z kterých jsme si mohli poskládat designový systém pro naši webovou aplikaci. Material Design 3 jsme zvolili, protože se jedná o moderní design, kde jsou již všechny vzory odzkoušené pro co nejlepší uživatelskou přívětivost. Využili jsme tedy jejich návrh barev, typografii, ikony a návrh komponent, které jsme implementovali v Reactu. Následně jsem sestavili vlastní layout, který lze vidět na obrázku 1.

Vlevo se nachází vedlejší navigační menu (komponenta Navigation rail), která slouží pro navigaci mezi hlavními stránkami a obsahuje komponentu Fab pro přidávání obsahu. Pro některé stránky jsme toto menu využili pro navigační šipku zpět, aby se uživatel mohl snadno přesunout na předchozí stránku. V horní části stránky se nachází navigační lišta pro základní operace a navigaci, jako jsou změna tmavého/světlého režimu, pro učitele archiv, uživatelská nastavení a odhlášení. Toto horní menu zobrazujeme na všech stránkách, kde nezabírá místo pro hlavní obsah.

Hlavní obsah se nachází ve zbytku stránky. Většinou se skládá z velkého nadpisu, který vystihuje aktuální stránku a pod ním se již nachází samotný obsah. V případě stránky konverzace jsme se inspirovali designovým návrhem Material Design 3 kit [12], který se skládá z levého menu, kde je seznam konverzací a na pravo se již nachází samotná konverzace. Podobný styl jsme zvolili i pro Ankety, kde jsme si design přizpůsobili vlastním potřebám.

3.2 Návrh databáze

Databáze je klíčová součást webové aplikace a slouží pro ukládání dat. Použili jsme knihovnu `mysql2` [13], která slouží pro připojení a práci s databází MySQL v Node.js. Pro navázání spojení slouží soubor `db.ts` ve složce `lib`, v kterém vytváříme pomocí metody `createPool` pool s danými vlastnostmi. Host určuje IP adresu serveru a port, na kterém portu je spuštěna, user a heslo slouží k zadání přihlašovacích údajů a nakonec se uvede název databáze. Primárním klíčem každé tabulky jsme zvolili sloupec s názvem `id` a typem `INT`, který je nastaven na `AUTO INCREMENT`, což znamená, že při přidání vždy zvýší hodnotu `id` o 1 a zaručuje tak unikátnost každého řádku. Pool funguje oproti `connection` následovně: vytvoří se pool s počtem spojení (výchozí je 10, ale lze změnit ve vlastnostech), tato spojení zůstávají otevřená a pokud potřebujeme se připojit k databázi, využijeme právě jedno spojení z poolu, poté co ho již nepotřebujeme, se automaticky vrátíme do poolu. Tento přístup zvyšuje výkon při práci s databází, což je dobré právě u aplikace, kterou využívá více uživatelů. Protože ve své aplikaci používáme jednu databázi, nám stačí používat jeden pool, proto využíváme návrhový vzor `singleton`, který vytvoří jeden sdílený pool a zamezí neustálému vytváření nových poolů, které mohou vést až k chybě `Too many connection`, kdy se vyčerpají veškerá spojení [14, 15].

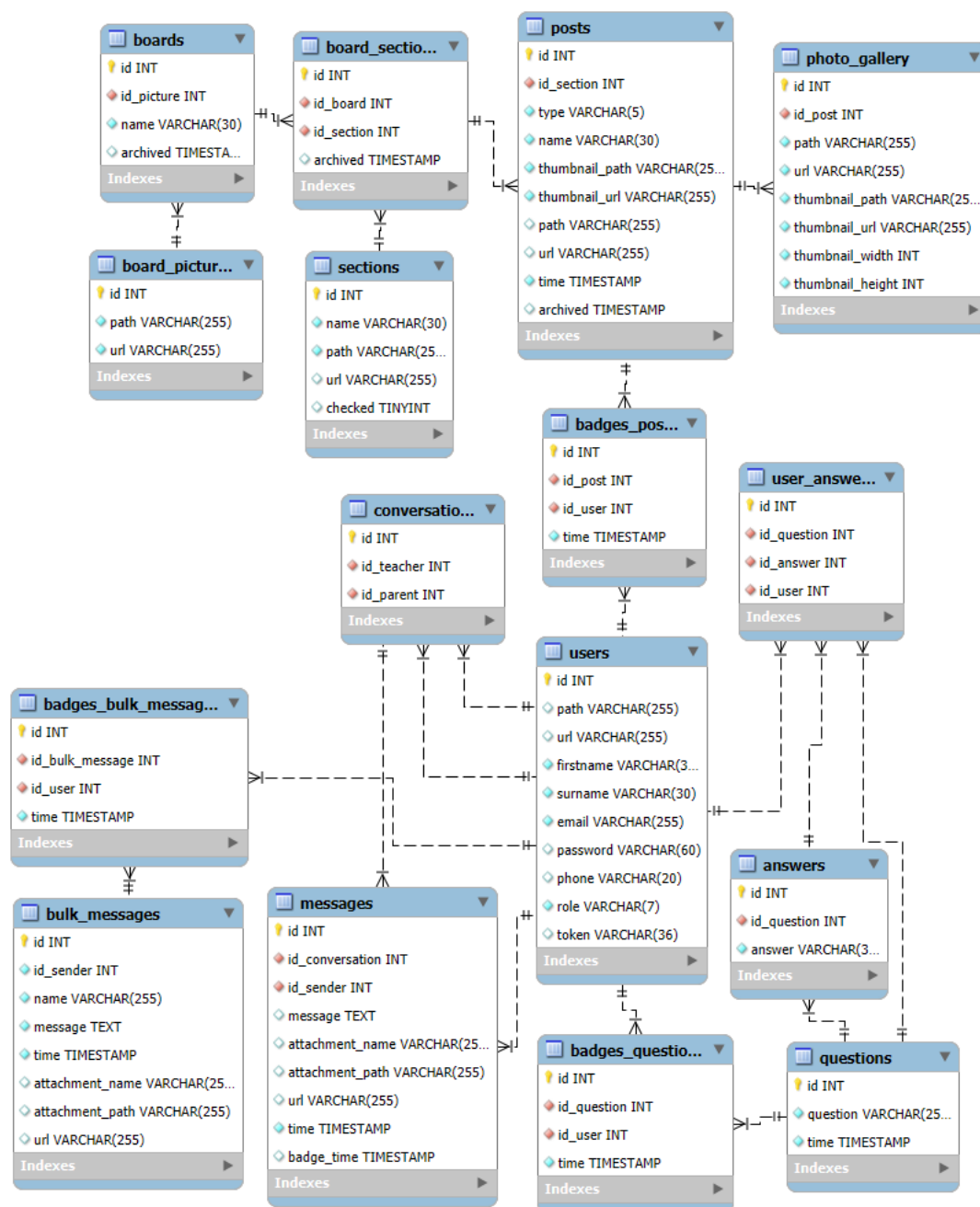
Pro samotné použití spojení v API stačí importovat dané spojení přes `import { pool } from "@lib/db"` a následně stačí již použít daný pool s danou metodou `execute` nebo `query`, dle případu užití. Následně se knihovna postará o získání spojení, provedení dotazu a vrácení spojení, což umožňuje `promise` verze knihovny s podporou asynchronních operací.

3.2.1 Transakce

Pokud provádíme zároveň více příkazů za sebou, může se stát, že by druhý příkaz selhal a první by se provedl a tím by se aplikace mohla dostat do nevalidního stavu. Tento problém řešíme pomocí transakcí, kdy se dané dotazy v transakci zapisují až po potvrzení transakce. Transakce také používáme v případech, kdy zapisujeme na disk a mohlo by se stát, že zápis selže a nastal by stejný stav jako v prvním případě. Abychom mohli implementovat transakce, tak z daného poolu musíme získat spojení přes metodu `getConnection`. Následně všechny dotazy provádíme na daném spojení. Pro zahájení transakce slouží metoda `beginTransaction` a po ní následují příkazy v dané transakci, které potvrzujeme metodou `commit`. V případě chyby vrátíme změny přes `rollback`. Na konci vždy vrátíme dané spojení do poolu metodou `release`.

3.2.2 Tabulky uživatelů

Všechny tabulky a propojení mezi nimi jsou zobrazeny v EER diagramu na obrázku 2. Pro ukládání uživatelů jsme vytvořili tabulku `users`, která se skládá z křestního jména `firstname` a příjmení `surname`, které jsme nastavili na typ



Obrázek 2: EER diagram databáze

`VARCHAR(30)`, který dává dostatečný prostor i pro dlouhá jména. Tabulka obsahuje také sloupec `password`, kam ukládáme hash hesla, který má vždy 60 znaků, proto jsem nastavil typ na `VARCHAR(60)`. V sloupci `role` ukládáme jednu ze dvou rolí `teacher` a `user`, které reprezentují učitele a uživatele jako je třeba rodič, sloupec jsme nastavili na typ `VARCHAR(7)` a je možné do něho v budoucnu ukládat více rolí. Další sloupce jsou `phone` pro telefonní číslo a `token` pro token, který slouží pro ověření uživatele při nastavení hesla. Sloupec `path` určuje adresu k profilovému obrázku a sloupec `url` adresu api, přes které je profilový obrázek přístupný uživatelům aplikace.

3.2.3 Tabulky nástěnek

Tabulka `boards` slouží pro ukládání nástěnek, obsahuje název `name` a příznak `archived`, zda je nástěnka archivována. Protože každá nástěnka může mít různý obrázek, vytvořili jsme tabulku `board_pictures`, která slouží pro uložení cesty k obrázku a cesty k api, přes které je obrázek dostupný. Tabulky jsou propojené přes cizí klíč na sloupci `id_picture`, který odkazuje na `id` v tabulce `id_pictures`. Podobně propojené jsou i tabulky `boards` a `board_sections`, kde ukládáme aktuální sekce k nástěnce. Zde je trochu jiná struktura než u nástěnek, protože každá sekce má daný obrázek, proto ukládáme název, `path`, `url` do jedné tabulky s názvem `sections`. V této tabulce je také sloupec `checked`, který slouží pro nastavení defaultních sekcí. Pokud je hodnota v tomto sloupci nastavená na 1, tak sloupec bude již předvybraný v seznamu při přidávání sekcí.

Příspěvky ukládáme do tabulky `posts`, která obsahuje název `name`, cestu `path` a `url` k miniatuře, dále cestu `thumbnail_path` a `thumbnail_url` k danému příspěvku, čas vytvoření `time` typu `TIMESTAMP` a příznak, zda je archivovaný. Jediná výjimka je u fotogalerie, kde z důvodu více dat na jeden příspěvek jsme vytvořili novou tabulku `photo_gallery`. V ní se nachází cizí klíč `id_post` na tabulku `posts` a dva další dva sloupce, které udávají šířku a výšku daného obrázku.

V tabulkách `boards`, `board_section` a `posts` indexujeme sloupec `name` pro rychlejší a efektivnější vyhledávání.

3.2.4 Tabulky dotazů

Pro uchování konverzací slouží tabulka `conversation`, kde jsou dva sloupce `id_teacher` a `id_parent`, které reprezentují konverzaci mezi učitelem a rodičem a odkazují na tabulku `users`. Na tuto tabulku je navázána tabulka `messages`, kde jsou uloženy jednotlivé zprávy, každá zpráva může obsahovat přílohu `attachment`, kde ukládáme její název, cestu a `url`. Dále v tabulce uchováváme `id_sender`, což je uživatel, který zprávu odeslal a čas `time`, kdy byla zpráva odeslána.

Další tabulka slouží pro hromadné zprávy, obsahuje `id` odesílatele, název zprávy, `message` zprávy, čas, kdy byla zpráva odeslána a přílohy, které mají stejnou strukturu jako v tabulce `messages`.

3.2.5 Tabulky anket

Pro ukládání otázek anket jsme vytvořili tabulku `questions`, kde je uložena daná otázka `question`, která může obsahovat až 255 znaků a aktuální čas, kdy byla vytvořena.

Odpovědi ukládáme do tabulky `answers`, kde je cizí klíč `id_question` na tabulku `questions` a odpověď, kterou jsme omezili na 30 znaků.

Pro uživatelské odpovědi jsme napsali tabulku `users_answers`, která obsahuje cizí klíče na tabulky `questions`, `answers` a `users`.

3.2.6 Tabulky a záznamy pro odznaky (badges)

Zbylé tabulky slouží pro odznaky, které uživatelům sdělují, že přibyl nebo se změnil aktuální záznam. Když jsou data ve formátu 1:n (jeden záznam, který si může zobrazit více uživatelů), tak jsou vytvářeny nové tabulky, pro post to je tabulka `badges_posts`, pro hromadné zprávy `badges_bulk_messages` a pro ankety tabulka `badges_question`. Pokud daný uživatel klikne na daný příspěvek, hromadnou zprávu nebo otázku, tak se do odpovídající tabulky vkládá záznam s jeho id, cizím klíčem na daný záznam a aktuálním časem. Pro zprávy, které jsou ve formátu 1:1 (mezi učitelem a rodičem) ukládáme informaci o čase zobrazení přímo do tabulky `signIn`.

3.3 Autentizace

Pro autentizaci jsme potřebovali spolehlivé a vyzkoušené řešení, které nám umožní si vytvořit vlastní přihlašovací formulář a poskytnout základní logiku, vybrali jsme proto knihovnu `NextAuth.js` [16].

Pro ověření používáme poskytovatele `Credentials` s vlastním formulářem, kde po ověření uživatele přes uživatelské jméno (e-mail) a heslo uživatele přesměrujeme na dashboard. Pro podporu použití funkcí `signIn` a `signOut` máme v souboru `auth.ts` v `pages` uvedenou trasu na vlastní formulář. V případě přístupu na zabezpečenou stránku bez ověření uživatele přesměrujeme přes `middleware`.

Pro ověřování jsme zvolili defaultní JWT strategii. JWT (JSON web token) je zašifrovaný token, který slouží pro ověření uživatele a ukládá se defaultně do zabezpečené session cookie. Funguje to tedy následovně: v souboru `auth.ts` máme definovanou funkci `authorize`, která slouží pro ověření uživatele, v této funkci voláme `API`, která ověří uživatele s databází a následně pošle informaci o jeho id a roli, které se uloží do tokenu. Pro získání dat pro front-end používáme `session callback`, který slouží pro uložení dat z tokenu do session, abychom uživatele mohli ověřovat na front-endu. Následně nám stačí používat v komponentě `useSession`, z které získáme `status` a `session`, ověříme, jestli `status` je `authorized` a jestli uživatel má danou roli. Ověření přes token pak používáme v `middleware` a k tomu slouží vestavěná funkce `getToken`.

Pro zabezpečení API Routes jsme použili pomocnou funkci `auth` doporučenou dokumentací [7], která zpřístupňuje ověření uživatele přes server funkci `getSession`.

Do databáze ukládáme hash se solí, který generujeme s pomocí knihovny `bcrypt` [17].

3.4 Implementace komponent

Ve webových aplikacích se často nachází znovu se opakující prvky jako jsou například tlačítka, ovládací prvky, kartičky, navigace a layouty. Abychom tyto prvky nemuseli neustále dokola implementovat, tak se nabízí jednoduché řešení vytvořit tento prvek pouze jednou a následně všude již používat jeho definici. Tento přístup výrazně zjednodušuje a zpřehledňuje kód a zároveň, pokud nás někdo požádá o změnu, tak nemusíme upravovat kód na někdy i desítkách míst, ale stačí nám jedna změna, která se provede všude, kde definici používáme. Tento problém jsme se rozhodli vyřešit pomocí knihovny `React` [4], kterou jsme vybrali právě pro implementaci těchto prvků, které nazýváme komponenty. V moderním `Reactu` se již upustilo od používání třídních komponent a v dnešní době se již používají funkční komponenty, jejichž syntaxe je mnohem jednodušší a přehlednější. Zároveň s nimi můžeme používat `React` hooky, což jsou funkce, které umožňují udržovat a měnit stav funkční komponenty.

3.5 Implementace REST API

REST API je základní součást této webové aplikace a tvoří backend část aplikace. Pro implementaci jsme použili `Next.js` Route Handlers.

3.5.1 Struktura

API jsme strukturovali do několika tras podle účelu (každá složka reprezentuje danou trasu). Základní trasy jsou `account`, `administration`, `archive`, `conversations`, `polls`, které se dále větví. Dále máme API pro přihlášení na trasu `login` a trasu pro inicializaci rodiče `init` (tato trasa slouží pouze pro inicializaci a z bezpečnostních důvodů bychom ji měli po použití smazat). Pro některé odznaky jsme vytvořili zvlášť trasy, abychom dodrželi správnou strukturu. Následně jsme pro všechny trasy, které zprostředkovávají přístup k souborům ve složce `private`, vytvořili zvlášť trasy `source`, čímž jsme zjednodušili jejich správu.

V projektu používáme dynamické segmenty trasy, které jsme vždy pojmenovali podle jejich účelu, abychom určili jasný význam, k čemu slouží. K těmto segmentům přistupujeme přes `object params`, který `Next.js` automaticky předává jako druhý argument vedle požadavku.

Z důvodu, že by uživatel mohl vkládat soubory se stejným názvem, tak pro každý soubor, který je nahrán přes API, vytváříme jedinečné `uuid`, které nastá-

víme jako jeho nový název. Využíváme k tomu JavaScript metodu `randomUUID` z rozhraní `Crypto`.

3.5.2 Databáze a typování

V každém API endpointu nejdříve načteme přístup k databázi přes `import { pool } from "@lib/db"`, kde pak přes objekt `pool` voláme metody `query` a `execute`.

Pro typování slouží typy z knihovny `mysql2` [13], které následně používáme, což jsou pro `SELECT` dotazy `RowDataPacket` a v případě dotazů jako jsou `INSERT`, `UPDATE`, `DELETE` `ResultSetHeader` [18].

3.5.3 Odpovědi

Odpovědi ve většině případů vracíme ve formátu JSON, ve kterém vracíme objekt v případě úspěchu s klíčem `message`, kterému nastavíme hodnotu například na `success` a v případě neúspěchu klíč `error`, kterému nastavujeme hodnotu podle dané chyby. Pro pojmenování hodnot jsme využili camel case konvenci. Dále vracíme status kód, v případě úspěchu se automaticky vrací kód 200, a proto ho explicitně neuvádíme. Přehled nejčastějších kódů s jejich názvy a popisem, které používáme:

- 200 OK – úspěšný požadavek, většinou vracíme `message` a `data` (pokud jsou požadována)
- 400 Bad Request – špatný požadavek, klient poslal nevalidní data
- 401 Unauthorized – neautorizovaný uživatel, uživatel není přihlášen nebo nemá dostatečná práva
- 404 Not Found – nenalezen zdroj, pokud nelze najít požadovaný zdroj
- 500 Internal Server Error – chyba serveru, je obecná odpověď při chybě na straně serveru

Následně na frontendu můžeme přes tyto status kódy a error hodnoty rozlišovat chyby. Pro obecnější chyby používáme status kódy, pokud chceme podrobnější chybu, tak si přečteme hodnotu `error` a následně vypíšeme odpovídající hlášku uživateli.

3.5.4 Odchycení chyb

Abychom odchytili veškeré chyby, které mohou vzniknout, tak používáme `try` a `catch`. Do `try` umístíme kód, který chceme ošetřit, pokud by se vyskytla chyba, tak se vyvolá výjimka, `catch` slouží k zachycení výjimky a dle paramtru v `catch` si můžeme vypsát informace o dané chybě do konzole příkazem `console.error(error)`. Následně klientovi vrátíme odpověď se status kódem 500 a univerzálním popisem v `error` hodnotě. Můžeme si definovat i vlastní

chyby například `throw new Error("notFoundPhoto")` a následně chybu zpracovat v `catch` bloku. V některých případech potřebujeme kód zavolat vždy na konci, k čemuž slouží `block finally`, který využíváme například při transakcích.

3.5.5 Zabezpečení

Pro zabezpečení importujeme pomocnou funkci `auth` z knihovny `NextAuth.js` [7], přes kterou přistupujeme ke `getSession`. Tuto funkci zavoláme a získáme danou `session cookie` z požadavku a zjistíme, jestli je uživatel přihlášen a případně ověříme jeho roli.

3.5.6 Poskytnutí dat

Přes API, které slouží pro přímé poskytnutí dat (trasy `source`) nevracíme data v JSON, ale přímo v těle HTTP odpovědi. Výhoda je, že k datům může přistupovat klient přímo přes daný API endpoint. Tato data mohou být jakýkoliv typ souborů. Pro poslání dat používáme JavaScript konstruktor `Response`. Jako první argument uvedeme data, která chceme poslat. Druhý argument je `json` objekt, v kterém můžeme nastavit hlavičky HTTP odpovědi. Nastavujeme například hlavičku `Content-Length`, kde udáváme velikost dat v těle. V každé odpovědi musíme nastavit typ dat v těle, které nastavujeme v hlavičce `Content-Type`. K zjištění, jakého typu jsou daná data, používáme knihovnu `mime` [19], která vrátí daný typ a tento typ následně vkládáme do hlavičky. Z důvodu, že knihovna může vrátit `null` hodnotu v případě, že by daný typ nezjistila, tak tento případ ošetřujeme a nastavujeme daný typ na `application/octet-stream`, což značí binární data s neznámým typem [20]. V případě, že chceme, aby daná data byla reprezentovaná jako příloha, tak nastavujeme hlavičku `Content-Disposition` na `attachment` a také zde nastavujeme název daného souboru.

3.6 Získávání dat

Aby stránka byla plně interaktivní, tak používáme React hooky jako jsou `useState` a `useEffect`, které lze používat pouze v klientských komponentách. Z toho důvodu vytváříme v klientských komponentách asynchronní funkce s předponou `load` pro načítání dat. K získání dat slouží JavaScript funkce `fetch`, jejíž první argument je URL adresa, kam se posílá HTTP požadavek. Druhý argument je nepovinný, a když ho neuvedeme, tak se automaticky použije metoda `GET`. Případně můžeme uvést objekt, kde lze nastavit danou metodu a hlavičku. Používáme také klíčové slovo `await`, aby se počkalo na odpověď z požadavku. Následně odpověď zpracováváme. Pokud vše proběhne v pořádku, vrátíme status 200 (ok) a daná data uložíme do stavu, pokud ne, vypíšeme chybovou hlášku. Celou tuto funkci vložíme do `useEffect`, kde můžeme do `dependency` pole (pole hodnot proměnných) vložit závislosti. V případě, že data načítáme pouze při prvním renderu, tak se uvádí pole prázdné.

3.7 Posílání, mazání a změna dat

Kromě získávání dat můžeme pomocí `fetch` funkce také data posílat. Nejdříve uvedeme URL API endpointu, kam data posíláme, a v druhém argumentu objekt, v kterém nastavíme metodu, kterou budeme používat. Pro posílání dat slouží metoda `POST`, pro změnu dat metoda `PUT` a pro mazání dat metoda `DELETE`. V případě metody `POST` a `PUT` můžeme data posílat a vkládat je do těla HTTP požadavku. V případě, že data posíláme jako JSON objekt, musíme použít metodu `JSON.stringify` a nastavit hlavičku `"Content-Type"` na `"application/json"`.

3.8 Nástěnky, sekce, příspěvky

Pro implementaci informační nástěnky jsme zvolili hierarchickou strukturu podobnou složkám, což zajišťuje přehlednost a jednoduché používání. První úroveň jsou nástěnky a druhá úroveň jsou sekce, do kterých vkládáme příspěvky. Tento přístup zajišťuje učitelům velkou flexibilitu s možností vše uspořádat dle vlastního uvážení.

Nejdříve jsme vytvořili komponentu `Card` (`karta`), jejíž základní funkcí je zobrazit obrázek, jméno a přesměrovat uživatele na další stránku. Učitelé navíc umožňují rozkliknout menu, které se ukáže při kliknutí na tlačítko s třemi tečkami, kde může přesunout danou kartu do archivu. Pro navigaci jsme vytvořili komponentu `Navigation`, která slouží pro přesměrování a zároveň zobrazuje aktuální cestu mezi nástěnkami a sekcemi.

V případě učitele se v levém menu nachází komponenta `Fab`, což je tlačítko s ikonkou plus, která přesměruje učitele na průvodce vytvořením. V případě více kroků v horním okraji zobrazujeme progress bar, který učitelé procentuálně zobrazuje, v jakém kroku se nachází. Vždy je nutné nejdříve určit název, který se vepisuje do komponenty `TextField`. Vstup musí být nenulový a maximálně 30 znaků, vstup kontrolujeme nejdříve na frontendu a z důvodu bezpečnosti i na backendu. Pokud uživatel zadá chybný vstup, tak nemá možnost pokračovat na další krok, ke kterému slouží tlačítko `Další` a vypíše se pod vstupním polem chybová hláška. Krokovač jsme implementovali přes stav, kde si ukládáme aktuální číslo kroku a podle něho zobrazujeme aktuální obsah.

3.8.1 Nástěnky a sekce

V případě nástěnky jsou celkem tři kroky, v prvním se nastaví název, v druhém obrázek a v třetím se ze seznamu vybere sekce. V druhém kroku je možné přidat vlastní obrázek, který se přidá do výběru obrázků, které lze použít pro nástěnky. V třetím kroku lze vybrat sekce a také vytvořit novou sekci s novým jménem a obrázkem, který se přidá do výběru sekcí. Zvolili jsme přístup, že každá sekce má daný obrázek, aby si uživatel vždy daný obrázek spojil s jejím účelem, což zvyšuje uživatelskou přívětivost. V posledním kroku se při kliknutí na tlačítko `Vytvořit` odešlou všechna data na API, kde je zpracujeme a vrátíme v pří-

padě chyby chybovou hlášku, nebo nástěnky vytvoříme a přesměrujeme učitele na hlavní stránku Nástěnky.

V případě přidání sekcí se zobrazí stránka se sekcemi, které lze do dané nástěnky přidat a kde lze také přidat novou sekci do výběru.

3.8.2 Příspěvky

V aplikaci lze přidávat šest druhů příspěvků. Při přidání učitel každému příspěvku v průvodci nastaví jméno a následně v případě souboru, PDF, videa a audia nahraje dané médium a při kliknutí na Vytvořit se odešle na API, kde data od uživatele zpracujeme, uložíme na disk, do databáze a klienta přesměrujeme na stránku s příspěvkem, kde příspěvek zobrazíme. Při vytvoření příspěvku vytváříme na disku hierarchickou strukturu s aktuální cestou k danému příspěvku, cesta začíná složkou `boards`, do které hierarchicky ukládáme `id` příslušné `boards`, `section` a `posts`, v kterém se již nachází daný příspěvek. Tento přístup nám umožňuje použít rekurzivní mazání.

3.8.3 Textové příspěvky

Pro implementaci textových příspěvků jsme zvolili `TipTap`. Editor je `headless`, což znamená, že nám poskytuje základní logiku a zároveň jsme si mohli vytvořit vlastní vzhled a tím dodržet design naší aplikace. Po vytvoření či uložení příspěvku ukládáme data ve formátu JSON, což je jeden z formátů, který editor podporuje. Kdykoliv chceme textový dokument zobrazit, nahrajeme tento JSON do editoru a zobrazíme ho. Neoprávněným uživatelům místo menu s úpravami zobrazíme název dokumentu a schováme tlačítko pro uložení.

3.8.4 Fotografie

Pro fotografie jsme vytvořili fotogalerii, kam lze nahrát jeden a více obrázků. Z důvodu optimalizace jsme omezili jednu fotogalerii na 100 fotek a v případě potřeby můžeme počet fotek navýšit. Aby si uživatel mohl prohlédnout náhledy a případně upravit fotky ještě před posláním, tak jsme využili knihovnu `react-images-uploading` [21], která zajišťuje základní logiku a zobrazení fotek na klientovi a následně jsme si vytvořili vlastní design pro jejich úpravu. Po nahrání fotek je přes form data odešleme na backend, kde je dále zpracováváme. Základní informace o názvu a miniatuře ukládáme do tabulky `posts`, následně informace o dané fotce a jejím náhledu do tabulky `photo_gallery`. Na disku ukládáme jednotlivé komprimované fotky do složky s příspěvkem. Jejich náhledy jsou pak uloženy ve vytvořené složce `thumbnails` a miniaturu pro daný příspěvek ukládáme do složky `thumbnail`. Pro zobrazení fotogalerie jsme využili knihovnu `yet-another-react-lightbox` [22] s knihovnou `react-photo-album` [23], která slouží pro zobrazení náhledů.

3.9 Komprese

Veškeré obrázky, které uživatel nahraje, prochází kompresí, kterou děláme z důvodu optimalizace, protože uživatel může nahrát moc velké obrázky, které by se následně dlouho načítaly a zabíraly zbytečně moc místa na disku. K tomu využíváme knihovnu `sharp` [24], jež disponuje funkcí `sharp`, která přijímá jako první argument `buffer`, kde je nahrán daný obrázek. Následně používáme metodu `resize`, která zmenší obrázek na rozměry, které se uvedou jako argument, následně používáme metodu `toFile`, která jako argument přijímá cestu k souboru. Obrázky prochází nejen kompresí, ale také měníme jejich velikost. Pro ikony jsme zvolili šířku 320 px, pro miniatury a náhledy 1280 px a pro obrázky ve fotogalerii 1920 px.

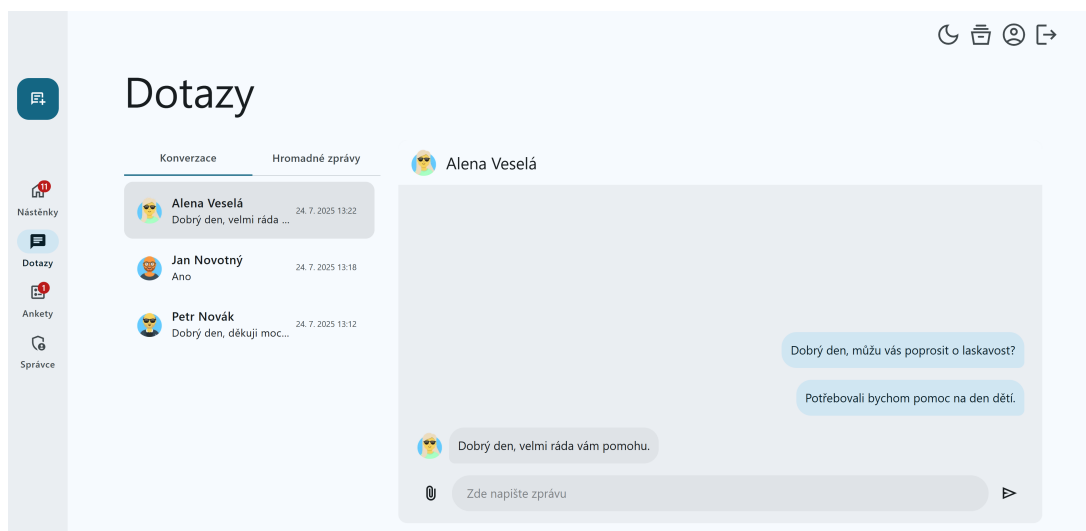
3.10 Vyhledávání

Vyhledávání je jednou z nejčastějších funkcí, kterou lze najít ve webových aplikacích. Vyhledávání jsme umožnili v příspěvcích, v sekcích i nástěnkách, což uživateli zrychluje přístup k daným příspěvkům, které zrovna hledá.

Pro vyhledávání jsme vytvořili komponentu `Search` a pro její implementaci jsme se inspirovali z oficiální podpůrných materiálů `Next.js` [25]. Komponenta přijímá dva argumenty `placeholder` a `isOpen`, kde `placeholder` je textový řetězec, který se zobrazí, pokud je vstupní pole prázdné, `isOpen` je stav, zda se má vyhledávání zobrazit, což je z důvodu mobilní verze, kdy vyhledávání zobrazujeme při kliknutí na ikonu vyhledávání. Nejdříve uživatel zapíše vstup přes `input`, kterému jsme nastavili typ `search`, tento vstup při změně nastavujeme jako argument funkce `handleSearch`, která vstup dále zpracovává. Následně používáme parametry vyhledávání, to znamená, že se vstup vepisuje do URL, z které následně vstup zpracováváme. Výhoda tohoto přístupu je, že si uživatel může danou adresu pro vyhledání zkopírovat do záložek a v budoucnosti ji kdykoliv načíst [25]. Pro přístup k parametru z URL slouží funkce `useSearchParams` a pro její manipulaci se používá konstruktor `URLSearchParams`. Například z URL ve tvaru `?query=video` (které vrátí hook `useSearchParams`) `URLSearchParams` získá parametr `video`. Následně tento parametr můžeme přes metodu `set` nastavit na vstup od uživatele, anebo případně parametr metodou `delete` z URL smazat. K propsání změn do URL se následně používáme metodu `replace` z `Next.js` hooku `useRouter`. Aby se hodnota z URL při prvním načtení projevila ve vstupu, tak je potřeba ještě nastavit `defaultValue` na aktuální parametr z URL.

3.11 Dotazy

Aby se mohl uživatel (rodič) dotázat učitele, tak při vytvoření uživatele vždy vytváříme novou konverzaci mezi uživatelem a rodičem. Tato konverzace se následně zobrazí na stránce Dotazy, která je zobrazena na obrázku 3, kde se nachází v levém sloupci seznam konverzací a v pravém sloupci se zobrazuje aktuální



Obrázek 3: Stránka s dotazy

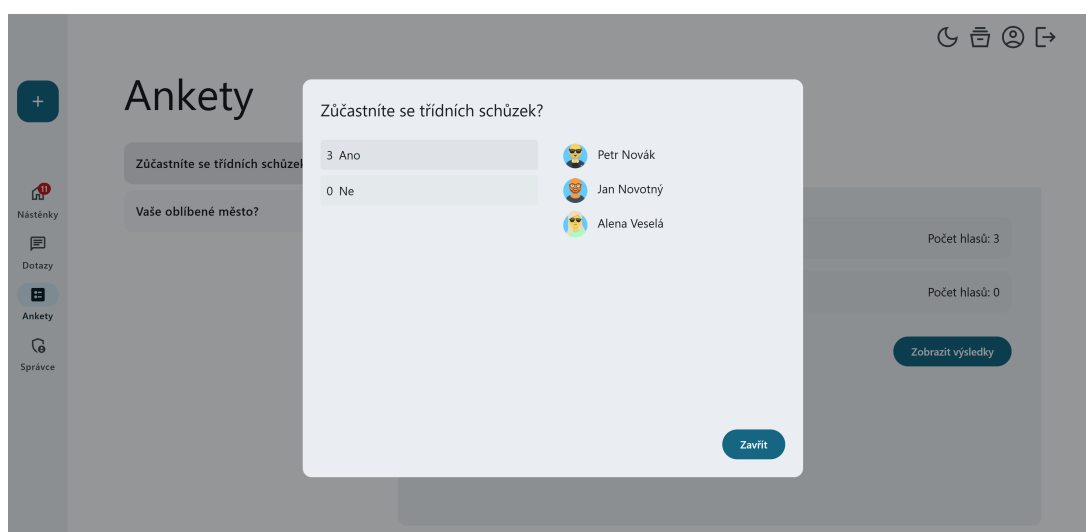
konverzace. Tento přístup nahrazuje klasickou potřebu vytvářet neustále nové konverzace nebo e-maily a zjednodušuje a zphřehledňuje rodiči práci. K napsání zprávy slouží vstupní pole a k odeslání tlačíko vpravo. Jako zprávu nebo se zprávou lze odeslat přílohu, ke které slouží tlačíko vlevo s ikonou přílohy. Následně se zprávy vždy řadí od poslední odeslané k té nejstarší, aby měl uživatel vždy přístup k té aktuální. Pro obnovování zpráv a konverzací máme nastavený v `useEffect` interval na 10 sekund. Pokud by byl požadavek na co nejkratší odezvu, tak by bylo možné implementovat `WebSocket`, ale vzhledem k počtu uživatelů a frekvencí dotazů to není potřeba. Jednotlivé konverzace a zprávy ukládáme do stavů, které při odeslání posíláme na backend, kde je následně zpracujeme.

Další užitečnou funkcí jsou hromadné zprávy, které může učitel poslat všem uživatelům aplikace. I k těmto hromadným zprávám lze přiložit přílohu a také název, který může být dlouhý až 255 znaků.

Pro zprávy i hromadné zprávy uživatele nechceme příliš omezovat ve velikosti vstupu, a proto používáme typ `TEXT`, který umožňuje uložit až 65 535 bytů na jednu zprávu. Případně bychom mohli nastavit limit přes typ `VARCHAR` a tím uživatele více omezit v počtu znaků, které může ve zprávě poslat.

3.12 Ankety

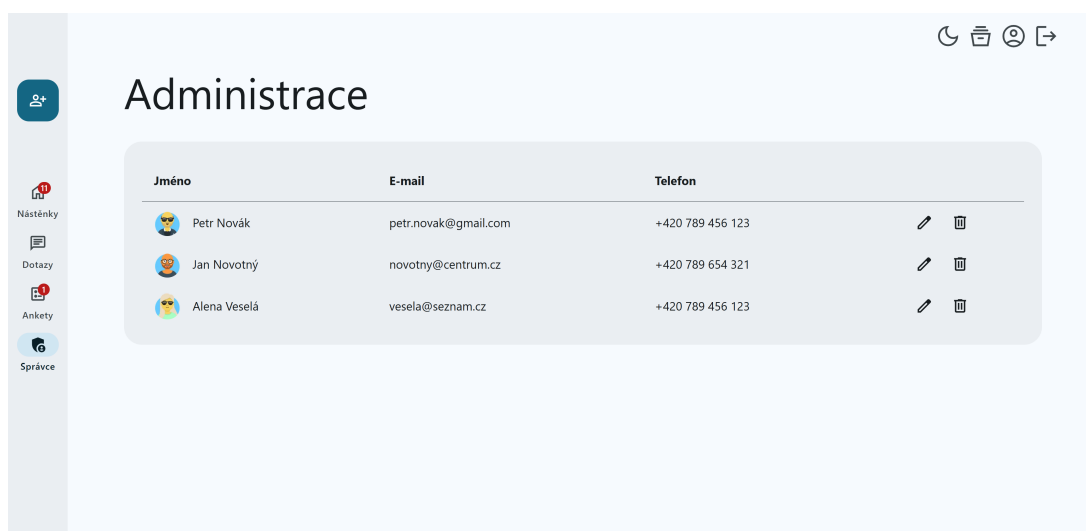
Abý učitel mohl získat rychlou zpětnou vazbu od uživatelů, tak jsme vytvořili ankety. Při vytvoření anket rodič zadá otázku, která může být dlouhá až 255 znaků a odpověď, kterou jsme omezili na 30 znaků. Odpovědi může být dvě a více, aby měl rodič alespoň dvě možnosti, z kterých může vybrat. Po vytvoření ankety učitel vidí název a u každé odpovědi počet hlasů. Pokud klikne na tlačítko Zobrazit výsledky, tak se mu ukáže přehledné dialogové okno, které je in-



Obrázek 4: Dialogové okno s výsledky na stránce ankety

spirováno článkem o UX designu [26]. Dialogové okno je zobrazeno na obrázku 4. Nalevo můžeme vidět jednotlivé odpovědi a napravo, kdo pro danou odpověď hlasoval, může si tak učitel přehledně prohlížet, jak hlasovali jednotliví uživatelé. Když uživatel hlasuje poprvé, tak se záznam vkládá a následně, pokud uživatel změní odpověď, tak v databázi hodnotu aktualizujeme.

3.13 Správa uživatelů



Obrázek 5: Stránka se správou uživatelů

Stránka se správou uživatelů se zobrazuje pouze učitelům a je vidět na obrázku 5. Pro správu uživatelů jsme zvolili tabulku, kde v každém záznamu zobrazujeme informace o uživateli. Učitel si tak může jednoduše dohledat informace o

rodiči. Po kliknutí na záznam může zobrazit velký náhled o daném uživateli. Učitelé umožňujeme dále měnit informace o rodiči, jako je jméno, příjmení, telefonní číslo. E-mailová adresa je spjatá s účtem, a proto tuto informaci měnit neumožňujeme. Dále rodič může resetovat heslo, po čemž aplikace odešle uživateli e-mail s odkazem na nastavení nového hesla. Dále může rodič uživatele smazat, čímž se smažou i veškeré konverzace s daným uživatelem, proto před těmito akcemi používáme dialogová potvrzovací okna, aby se náhodou nestalo, že by se překliknul.

3.13.1 Vytvoření uživatele

Při vytvoření uživatele validujeme jednotlivé vstupy. Abychom ověřili, zda se jedná o validní e-mail, využíváme knihovnu `validator` [27]. Tato knihovna disponuje objektem `validator` a metodou `isEmail`, která přijímá jako argument danou e-mailovou adresu. Po ověření nám vrátí logickou hodnotu, dle které můžeme v případě nevalidní hodnoty zobrazit chybovou hlášku. Pokud je e-mail validní, tak ho s dalšími daty pošleme na API, kde data dále zpracováváme a znovu ověřujeme. Aby bylo nastavení hesla pro uživatele transparentní a bezpečné, zvolili jsme přístup, kdy uživateli posíláme e-mail s odkazem na stránku s přihlášením. Pro odesílání e-mailu jsme využili knihovnu `nodemailer` [28] a pro její nastavení jsme vytvořili funkci v souboru `mailer.ts` ve složce `lib`. V této funkci vytváříme `uuid`, které bude sloužit jako token pro ověřování, následně vytvoříme link s adresou na stránku s nastavením hesla a do parametru `?token=` vložíme právě daný token. Tento link bude sloužit uživateli k přihlášení a při přihlášení budeme token ověřovat s databází, kam ho při vytvoření uživatele uložíme. Následuje samotná funkce pro nastavení adresy a portu SMTP serveru. Nastavujeme hodnotu `secure` pro nezašifrované spojení na `false` a pro zašifrované na `true`. Pro testování odesílání e-mailu jsme zvolili službu `Ethereal` [29], která používá nezašifrované spojení, v případě nasazení by se použila jiná služba, která by měla být vždy zašifrovaná. Dále nastavujeme přihlašovací údaje k SMTP serveru. Nakonec pošleme přes asynchronní funkci e-mail s danými údaji a odkazem. Po rozkliknutí odkazu se uživatel přesměruje na stránku pro nastavení hesla, kde heslo musí uvést dvakrát, aby se nestalo, že by se například překliknul. Po odeslání ověříme z parametru `token` s databází a uživatele přesměrujeme na stránku s přihlášením a v případě chyby vypíšeme danou chybovou hlášku.

Dále ověřujeme a formátujeme zadané telefonní číslo. Pro ověření jsme použili knihovnu `libphonenumber-js` [30] a nastavili jako defaultní české telefonní číslo. Knihovna tedy automaticky přes funkci `parsePhoneNumber` číslo naformátuje a přidá českou předvolbu, v případě jiné než výchozí předvolby se před telefonní číslo uvede daná předvolba a knihovna číslo naformátuje podle pravidel dané země.

3.14 Nastavení uživatele

Nastavení uživatele je stránka, která je přístupná přes horní menu, v případě mobilní verze přes hamburger menu. Jak název napovídá, stránka slouží pro uživatelská nastavení. Umožňujeme zde uživateli změnit obrázek. Vytvořili jsme proto dialogové okno, kde lze daný obrázek nastavit a při nahrání obrázku zobrazujeme jeho náhled, který ukládáme do dočasné URL adresy na klientovi přes objekt v JavaScriptu `URL.createObjectURL`. Po nahrání profilové fotky se zobrazí aktuální obrázek. Z důvodu, že používáme API, které zpřístupňuje danému uživateli obrázek, tak aby webový prohlížeč zobrazil okamžitě aktuální obrázek a nenahrával obrázek, který má uložený v cache paměti, tak po získání dat přidáváme k dané URL parametr `uuid` s jedinečným identifikátorem.

Dále na stránce umožňujeme změnit uživatelská data jméno, příjmení a telefonní číslo, které formátujeme stejně jako v případě správy uživatelů přes knihovnu `libphonenumber-js` [30].

V poslední části stránky si může uživatel změnit heslo, nejdříve je nutné zadat původní heslo a následně nové heslo, které musí potvrdit. V případě zadání špatného původního hesla, nebo pokud se nová hesla neshodují, uživateli vypíšeme chybovou hlášku.

3.15 Mobilní verze

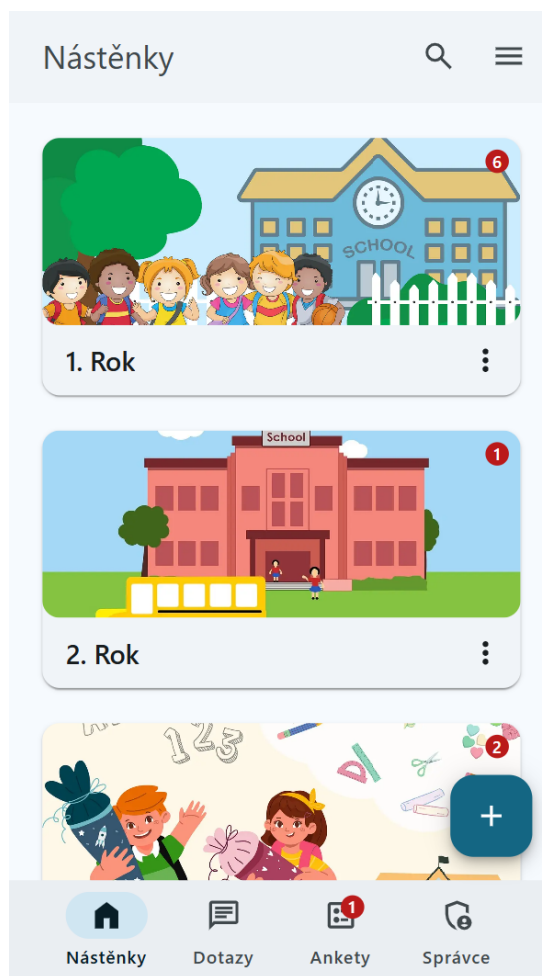
Dnes již mnoho uživatelů používá webové aplikace na mobilních zařízeních a tabletech, a proto je aplikace plně responzivní, což znamená, že se automaticky přizpůsobuje šířce obrazovky.

Responzivní webovou stránku jsme implementovali převážně přes Tailwind CSS, kde se vždy před danou utilitu uvedou prefixy, které fungují jako breakpointy media queries. Nejdříve se tedy definují všechny utility pro mobilní verzi a následně se podle velikosti obrazovky udávají prefixy jako jsou `sm:`, `md:`, `lg:`, `xl:`, `2xl:`. Tomuto přístupu se říká *mobile first*, což má výhody především v rychlejším načítání na mobilních zařízeních.

Zároveň, pokud nechceme nějakou část zobrazit na mobilním zařízení, používáme utilitu `hidden`, která nastaví `display: none;` a následně zviditelnujeme přes nastavení `display` vlastnosti s použitím utilit jako jsou nejčastěji `block`, `inline`, `inline-block`, `grid` a `flex`.

Pro podmíněné vykreslování jsme zvolili knihovnu `react-responsive` [31], která umožňuje vytvořit proměnné dle breakpointů. Můžeme díky tomu například ve funkcích určit, co se má vykreslit pouze pro počítačovou verzi.

Pro mobilní zařízení jsme vytvořili nové hamburger menu, které se nachází vpravo nahoře, jak je vidět na obrázku 6. Po rozkliknutí se objeví pravé menu, v kterém se nachází navigační prvky jako jsou uživatelské nastavení, pro učitele archiv, změna tmavého/světlého režimu a odhlášení, které lze nalézt v horním menu v počítačové verzi. Horní menu jsme využili pro zobrazení nadpisu dané stránky, na některých stránkách zobrazujeme také šipku zpět. Také jsme přidali ikonu pro vyhledávání v odpovídajících stránkách. Levé menu jsme pře-



Obrázek 6: Mobilní verze

sunuli na spodní část obrazovky a tlačítko Fab jsme umístili vpravo dole nad ním a zpřístupnili tak veškeré ovládací prvky. U sekce dotazy a ankety jsme vše přizpůsobili pro co nejsnadnější používání, proto pravou část zobrazujeme až po kliknutí, a poté schováváme spodní menu a tlačítko Fab, které nejsou v daný moment potřeba. Zpět na hlavní stránku se lze dostat přes šipku zpět v horním menu. V správě uživatelů vždy omezuje počet sloupců v tabulce podle šířky obrazovky, díky tomu je vždy vidět celá tabulka, v případě zobrazení všech informací lze přejít do náhledu uživatele.

3.16 Bezpečnost

Bezpečnost webových aplikací je naprosto zásadní. Z důvodu, že jsou dostupné komukoliv v síti, je nutné aplikaci zabezpečit. Zvolili jsme proto několik osvědčených přístupů pro zabezpečení naší aplikace.

Pro zamezení přístupu na stránky, kam uživatel nemá právo vstoupit, pokud není například přihlášen nebo nemá odpovídající roli, jsme využili middleware,

který umožňuje spustit kód ještě před dokončením požadavku. Všechna pravidla jsou uvedena v k tomu určenému souboru `middleware.ts`, kde nejdříve získáme token přes funkci `getToken` (funkce z knihovny `NextAuth.js` [16]), z které zjistíme, jestli existuje ověřený token pro daného uživatele, tedy zda je uživatel přihlášen, pokud není, tak ho přesměrujeme na stránku s přihlášením. Dále zjistíme z tokenu, zda daný uživatel není učitel (role `teacher`) a pokud není, tak porovnáme, zda chce přistoupit na jednu ze stránek, které jsou vyhrazeny pouze pro učitele, pokud by chtěl přistoupit na jednu z tras, kam nemá přístup, přesměrujeme ho na stránku s nástěnkami.

Jedním z častých útoků je SQL injection, kdy útočník chce přes nechráněný vstup dělat neoprávněné změny v databázi. Z toho důvodu jsme se rozhodli pro parametrizované dotazy, které umožňuje knihovna `mysql2` [13]. To znamená, že pokud potřebujeme do dotazu vložit nějakou proměnnou, nevkládáme ji přímo, ale vytváříme nejdříve pole, kam uložíme veškeré proměnné a následně v samotném příkazu využijeme zástupný symbol `?` za každou proměnnou. Následně je nutné použít metodu `execute` a té předáme jako první parametr sql příkaz a jako druhý parametr pole s hodnotami a metoda se postará o bezpečné provedení příkazu.

Dalším problémem by mohlo být, kdyby uživatel mohl vkládat nulový nebo neplatný vstup a zároveň když překročí velikost vstupního pole, což by v aplikaci nebylo žádoucí a zároveň mohlo vést k různým chybovým stavům. Proto v API kontrolujeme, zda nedostáváme od uživatele neplatné hodnoty nebo zda nepřekročil limit znaků pro daný vstup.

3.17 Rozšíření

Mnoho aplikací, které byly vytvořeny, již po několika letech je zastaralých a běží na starých verzích, které mohou mít mnoho bezpečnostních chyb a často nelze je dále rozšiřovat. Z toho důvodu jsme zvolili nejmodernější technologie s možností co největší škálovatelnosti a rozšiřitelnosti.

Aplikaci můžeme dále vyvíjet a rozšiřovat. Můžeme například přidat další možnosti do příspěvku nebo vytvořit novou funkcionalitu. Vždy nám stačí přidat komponentu, vytvořit novou trasu a API endpoint. Také můžeme stáhnout další knihovny, kterých je mnoho, a které nám mohou pomoci aplikaci dále rozšiřovat o další skvělé funkce. Díky REST API můžeme vytvořit také nativní mobilní a desktopovou aplikaci.

Na pozadí běží také `Node.js`, který je znám svojí velkou škálovatelností [5]. Pokud by se nám aplikace dále rozšiřovala mezi lidi a potřebovali bychom vyšší výkon, můžeme aplikaci jak vertikálně i horizontálně škálovat, aby zvládla velký počet požadavků.

4 Porovnání s nástrojem Padlet

Padlet je nástroj, který umožňuje přidávat do nástěnky různé příspěvky a nasdílet ji přes odkaz uživatelům. Její omezení je především v jejím účelu, kdy slouží pro živou spolupráci mezi několika uživateli. Proto není úplně vhodná jako informační nástěnka pro trvalé uložení dat. Rozhodli jsme se proto jít jinou cestou a vytvořit plnohodnotnou informační nástěnku s hierarchickou strukturou. V Padletu také není možné spravovat dané uživatele, lze jim nasdílet nástěnku, ale již není možné mít seznam uživatelů se základními údaji, jako je třeba telefonní číslo, které často učitel potřebuje velmi rychle, proto tyto informace zobrazujeme v přehledné správě uživatelů. V Padletu je sice možné vytvořit provizorní anketu v nástěnce, kde se na ní mohou dotazovat, ale již neexistuje přehledné jedno místo, kde jsou dostupné všechny ankety. Sice v Padletu lze okomentovat danou nástěnku, ale již není možná přímá komunikace mezi uživateli.

Padlet je jeden z mnoha nástrojů, které mají jistý účel, ale nejsou vhodné jako plnohodná informační nástěnka právě z důvodu jejich zaměření. Těchto nástrojů je mnoho, ale pro učitele to znamená používat velké množství takových aplikací a velice rychle se v nich ztrácejí jak samotní učitelé, tak i rodiče. Proto jsme vytvořili webovou aplikaci, která tento problém řeší a poskytuje učitelům komplexní nástroj pro jejich výuku.

Závěr

V bakalářské práci jsme popsali komplexní řešení webové aplikace, která bude sloužit učitelům pro jejich výuku. Vytvořili jsme přehledného průvodce pro vytváření nástěnek a výběru sekcí. Umožnili jsme přidání několika druhů příspěvků, které učitel běžně potřebuje a přidali jsme vyhledávání jak v příspěvcích, tak v sekcích a nástěnkách. Vytvořili jsme přehlednou správu uživatelů, která nejen poslouží účelům administrace, ale také jako informační tabulka pro základní informace, které učitel potřebuje o rodičích. Pro dotazy jsme implementovali konverzace, kde lze kromě zpráv posílat i přílohy. Dále jsme učiteli umožnili posílat hromadné zprávy pro všechny uživatele a vytvářet ankety, kde se může dotazovat. Vše jsme vytvořili v moderním a plně responzivním designu, který se bude uživatelům dobře používat. Díky její struktuře a použití nových technologií lze aplikaci dále vyvíjet a rozšiřovat o další funkce.

Conclusions

In my bachelor's thesis, I described a comprehensive solution for a web application that will serve teachers for their teaching. I created a clear guide for creating bulletin boards and selecting sections. I enabled adding several types of posts that teachers commonly need, and I added search in posts, sections, and bulletin boards. I created a clear user management that will not only be used administration purposes, but also as an information table for basic information that teachers need about parents. For questions, I implemented conversations that allows sending not only messages but also attachments. I also enabled teachers to send bulk messages to all users and create polls where they can ask questions. I created everything in a modern and fully responsive design that users will use well. Thanks to its structure and the use of new technologies, the tool can be further developed and expanded with additional functions.

A První příloha

Text první přílohy

B Druhá příloha

Text druhé přílohy

C Obsah elektronických dat

Na samotném konci textu práce je uveden stručný popis obsahu elektronických dat odevzdaných v systému katedry informatiky spolu s textem. Tato data jsou nedílnou součástí práce a tvoří (datovou) přílohu textu práce. Povinné položky struktury dat jsou:

text/

Adresář s textem práce ve formátu PDF, vytvořený s použitím závazného stylu KI PřF UP v Olomouci pro závěrečné práce, včetně všech (textových) příloh, a všechny soubory potřebné pro bezproblémové vytvoření PDF dokumentu textu (případně v ZIP archivu), tj. zdrojový text textu a příloh, vložené obrázky, apod.

README.*

Textový soubor (s příponou např. `.txt`) s informacemi o opakovatelném způsobu použití ostatních dat práce – typicky plně reprodukovatelný co nejúplnější funkční postup zprovoznění software vytvořeného v rámci práce, tzn. jeho případné instalace/nasazení a spuštění, včetně uvedení všech požadavků pro bezproblémový provoz; za zprovoznění software se nepovažuje zpřístupnění (např. po Internetu) již někde zprovozněného software.

Adresáře a soubory s veškerými ostatními autorskými daty práce (případně v ZIP archivu) – typicky spustitelné a další soubory software vytvořeného v rámci práce potřebné pro bezproblémový provoz software, případně jeho instalační program, a kompletní zdrojové texty software a další data nutná pro plně reprodukovatelné korektní vytvoření spustitelných souborů.

Dále mohou data obsahovat například:

- ukázková a testovací data použitá v práci nebo pro potřeby posouzení práce v rámci její obhajoby,
- položky bibliografie v elektronické podobě, příp. jiná relevantní literatura a dokumentace vztahující se k práci,

- cizí data (software) potřebná pro bezproblémové použití autorských dat práce (software), která nejsou standardní součástí předpokládaného (softwareového) vybavení uživatele.

U veškerých cizích obsažených materiálů jejich zahrnutí dovolují podmínky pro jejich veřejné šíření nebo přiložený souhlas držitele práv k užití. Pro všechny použité (a citované) materiály, u kterých toto není splněno a nejsou tak obsaženy, je uveden jejich zdroj, např. webová adresa, v bibliografii nebo textu práce nebo souboru README.*.

Literatura

- [1] Frost, Brad. *Chapter 2: Atoms: Atomic Design* [online]. [cit. 2025-7-25]. Dostupný z: <https://atomicdesign.bradfrost.com/chapter-2/>.
- [2] Inc., Tailwind Labs. *Rapidly build modern websites without ever leaving your HTML: Tailwind CSS* [online]. 2025 [cit. 2025-7-25]. Dostupný z: <https://tailwindcss.com/>.
- [3] Microsoft. *TypeScript for JavaScript Programmers: TypeScript Documentation* [online]. 2025 [cit. 2025-7-25]. Dostupný z: <https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html>.
- [4] Meta. *Quick Start: React Documentation* [online]. 2025 [cit. 2025-7-25]. Dostupný z: <https://react.dev/learn>.
- [5] Foundation, OpenJs. *About Node.js®* [online]. 2025 [cit. 2025-7-25]. Dostupný z: <https://nodejs.org/en/about>.
- [6] Vercel, Inc. *Next.js Docs: Next.js Documentation* [online]. 2025 [cit. 2025-7-25]. Dostupný z: <https://nextjs.org/docs>.
- [7] Collins, Iain. *getSession: NextAuth.js Documentation* [online]. 2025 [cit. 2025-8-2]. Dostupný z: <https://next-auth.js.org/configuration/nextjs#getSession>.
- [8] Google. *Material Design* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://m3.material.io/>.
- [9] Google. *Material Theme Builder* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://material-foundation.github.io/material-theme-builder/>.
- [10] Building Robust Web, The Architect's Guide to; Apps, Mobile. *Next.js Docs* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://medium.com/@samwocks/the-architects-guide-to-building-robust-web-and-mobile-apps-fcf771a53625>.
- [11] IBM. *What is a REST API?: IBM Topics* [online]. 2025 [cit. 2025-7-26]. Dostupný z: <https://www.ibm.com/think/topics/rest-apis>.
- [12] Google. *Material Design 3 kit* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://www.figma.com/community/file/1035203688168086460>.
- [13] Sidorov, Andrey. *MySQL2: MySQL2 Documentation* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://sidorares.github.io/node-mysql2/docs>.
- [14] Hai, Truong Trong. *“globalThis”, “declare global” and the solution of Singleton Prisma client* [online]. 2024 [cit. 2025-8-3]. Princip singletonu platí nejen pro Prisma, ale i další klienty. Dostupný z: <https://medium.com/@truongtronghai/globalthis-declare-global-and-the-solution-of-singleton-prisma-client-7706a769c9d3>.

- [15] Farley, Brian. *The Singleton Design Pattern* [online]. 2024 [cit. 2025-8-3]. Dostupný z: <https://www.farleythecoder.com/blog/design-patterns/singleton-pattern>.
- [16] Collins, Iain. *NextAuth.js: NextAuth.js Documentation* [online]. 2025 [cit. 2025-8-2]. Dostupný z: <https://next-auth.js.org/>.
- [17] bcrypt. *node.bcrypt.js: bcrypt Documentation* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://github.com/kelektiv/node.bcrypt.js>.
- [18] Sidorov, Andrey. *Using MySQL2 with TypeScript: MySQL2 Documentation* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://sidorares.github.io/node-mysql2/docs/documentation/typescript-examples>.
- [19] Kieffer, Robert. *Mime: Mime Documentation* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://github.com/broofa/mime>.
- [20] MDN. *Discrete types* [online]. 2025 [cit. 2025-8-3]. Dostupný z: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/MIME_types.
- [21] Toan, Tran Van Vu. *react-images-uploading: react-images-uploading Documentation* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://github.com/vutoan266/react-images-uploading/tree/master>.
- [22] Danchenko, Igor. *Yet Another React Lightbox: Yet Another React Lightbox Documentation* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://yet-another-react-lightbox.com/documentation>.
- [23] Danchenko, Igor. *React Photo Album: React Photo Album Documentation* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://react-photo-album.com/documentation>.
- [24] Fuller, Lovell. *sharp: sharp Documentation* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://github.com/lovell/sharp>.
- [25] Vercel, Inc. *Adding Search and Pagination: Next.js Documentation* [online]. 2025 [cit. 2025-7-28]. Dostupný z: <https://nextjs.org/learn/dashboard-app/adding-search-and-pagination>.
- [26] Sharma, Tanvi. *Designing a poll feature to increase user engagement* [online]. [cit. 2025-8-3]. Dostupný z: <https://contra.com/p/mWklLEEk-designing-a-poll-feature-to-increase-user-engagement>.
- [27] validator.js. *validator.js: Validator.js Documentation* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://github.com/validatorjs/validator.js>.
- [28] Nodemailer. *Nodemailer: Nodemailer Documentation* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://nodemailer.com/>.
- [29] Ethereum. *Ethereal* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://ethereal.email/>.

- [30] Kuchumov, Nikolay. *libphonenumber-js: libphonenumber-js Documentation* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://gitlab.com/catamphetamine/libphonenumber-js>.
- [31] Schoffstall, Eric. *react-responsive: react-responsive Documentation* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://github.com/yocontra/react-responsive>.
- [32] Sidorov, Andrey. *Prepared Statements: MySQL2 Documentation* [online]. 2025 [cit. 2025-8-3]. Dostupný z: <https://sidorares.github.io/node-mysql2/docs/examples/queries/prepared-statements>.